# A Rule-Based Natural Language Understanding Module for a Domain-Constrained Restaurant Dialogue System

**Tabish Izhar**
Matriculation Number: 2161533
DS-Proj-M: Master Project Dialogue Systems
Faculty of Information Systems and Applied Computer Sciences
Otto-Friedrich-University of Bamberg

*Submission date: February 20, 2026*

## Abstract

This project proposes the design and development of a rule-based Natural Language Understanding (NLU) module as part of a Cambridge-centric and domain-specific restaurant recommendation chatbot. The user may query restaurant information using natural language queries, and these queries are then converted into structured database queries. The proposed NLU module will cater to 13 different dialogue intents and will be able to extract various entities from a single query. The proposed system will be developed as a modular system with various modules including NLU, dialogue state management, database query management, and response generation. The functionality of the proposed system will be tested using various dialogue scenarios. The proposed system will demonstrate the effectiveness of using a rule-based system as part of domain-constrained chats.

## Contents

**1 Introduction** 1

**2 Related Work** 2

**3 System Architecture** 2
3.1 Overall Architecture . . . 2
3.1.1 User Input Interface . . . 2
3.1.2 Natural Language Understanding (NLU) Module . . . 2
3.1.3 Dialogue State Update Component . 2
3.1.4 Dialogue Manager . . . 2
3.1.5 Restaurant Database Query . . . 2
3.1.6 Template-Based Response Generator 2
3.2 Data Flow and Processing Steps . . . 3
3.2.1 Input Preprocessing . . . 3
3.2.2 Intent Classification . . . 3
3.2.3 **Entity Extraction (Slot Filling)** . . 3
3.2.4 State Update . . . 3
3.2.5 Database Query Formation . . . 3
3.2.6 Response Generation . . . 3

**4 Natural Language Understanding Module** 3
4.1 Preprocessing . . . 3
4.2 Intent Classification . . . 4
4.2.1 Advantages . . . 4
4.2.2 Limitations . . . 5
4.3 Entity Extraction and Slot Filling . . . 5
4.4 Design Rationale . . . 5

**5 Dialogue Management and Database Integration** 5
5.1 Dialogue State Management . . . 5
5.1.1 Database Query Generation . . . 6
5.1.2 Template-Based Response Generation 6

**6 Functional Testing and Validation** 6
6.1 Testing Methodology . . . 6
6.2 Representative Test Scenarios . . . 6
6.2.1 Scenario 1: Greeting Detection . . . 7
6.2.2 Scenario 2: Multi-Slot Query . . . 7
6.2.3 Scenario 3: Alternative Request . . . 7
6.2.4 Scenario 4: Information Request . . 7
6.2.5 Scenario 5: Restart Operation . . . 7
6.3 Discussion of Evaluation Approach . . . 7

**7 Challenges and Limitations** 7
7.1 Linguistic Variability and Synonym Handling 7
7.2 Sensitivity to Input Variations . . . 7
7.3 First-Match Intent Strategy . . . 7
7.4 Single-Intent Constraint . . . 8
7.5 Scalability . . . 8
7.6 Domain Dependency . . . 8

**8 Conclusion and Future Work** 8

## 1 Introduction

Conversational agents are becoming an important interface to structured information. The dialogue system allows users to interact with software systems in an intuitive manner. The dialogue system has reduced the need to interact with software systems using command-based inputs. The early conversational systems, such as ELIZA (Weizenbaum, 1966), showed the possibility of rule-based dialogue interaction. The newer dialogue systems are also using statistical and neural-based natural language understanding techniques (Jurafsky and Martin, 2023; Young et al., 2013).

Despite the achievements made in machine learning, deterministic systems still have a place in environments with tight domain constraints, i.e., environments with a small vocabulary and user intentions that are clearly defined. The deterministic system provides transparency, predictability, and full control for the developer. In many real-world tasks, such as domain-specific information retrieval, a rule-based NLU module can provide

reliable performance without requiring large annotated datasets.

This project seeks to develop a natural language understanding module based on a set of rules for a chatbot that provides restaurant recommendation services. The chatbot will allow users to ask for restaurant details using natural language, which will be processed into a structured format that identifies the intent and the relevant slot values. This will be used by a dialogue management system that is integrated with a restaurant database.

The proposed system is based on a modular design that includes an NLU module, a dialogue state update module, a database query interface module, and a template-based response generation module. The key contribution of this research work is the implementation and evaluation of the proposed deterministic pattern-matching-based NLU module that can process multiple entities in a single user query.

## 2 Related Work

Dialogue systems have been extensively researched in the areas of Natural Language Processing (NLP) and Artificial Intelligence. Rule-based dialogue systems like ELIZA (Weizenbaum, 1966) were purely based on rule-based pattern matching techniques for simulating a conversation. Such systems proved that linguistic rules and templates could be used to create the illusion of understanding without resorting to semantic modeling.

Conventional dialogue systems are generally broken down into three primary components: Natural Language Understanding (NLU), Dialogue Management, and Natural Language Generation (NLG) (Young et al., 2013). In rule-based systems, the NLU module is generally based on grammars, keyword matching, or regular expressions for identifying the user intent and relevant entities. Such systems are deterministic and interpretable but can be challenging when it comes to linguistic variability and ambiguity.

However, with the advent of statistical approaches, the classification of intent and recognition of entities has increasingly been modeled using machine learning algorithms such as Support Vector Machines, Conditional Random Fields, and more recently, neural networks such as recurrent networks and transformer networks (Jurafsky and Martin, 2023; Devlin et al., 2019). While these models enhance generalization capabilities and resistance to paraphrasing, they require labeled training data and introduce uncertainty in the form of probabilities into the prediction process.

Despite the effectiveness of machine learning algorithms, rule-based systems are still valid in domain-specific applications where the number of intents and entities is small and well-defined. In domains such as task-oriented dialogue systems, where the domain is constrained, rule-based systems can provide high accuracy, interpretability, and ease of debugging (Young et al., 2013). Moreover, they do not require large amounts of labeled training data, which may not be feasible in small-scale academic or research projects.

The system described in this work uses a rule-based approach to NLU that is suited for a limited domain of restaurant recommendations. The system relies on regular expression pattern matching and entity vocabularies to ensure transparency, modularity, and predictable system behavior.

## 3 System Architecture

This section outlines the general architecture of the restaurant recommendation chatbot and the interaction between the main components. The system is based on a modular pipeline architecture, which is typical for task-oriented dialogue systems (Young et al., 2013), and breaks down the language understanding, dialogue state tracking, database access, and response generation into separate components.

### 3.1 Overall Architecture

The chatbot is structured as a sequential processing pipeline consisting of the following components:

#### 3.1.1 User Input Interface

#### 3.1.2 Natural Language Understanding (NLU) Module

#### 3.1.3 Dialogue State Update Component

#### 3.1.4 Dialogue Manager

#### 3.1.5 Restaurant Database Query

#### 3.1.6 Template-Based Response Generator

The data flow can be summarized as follows: User Input
→ NLU (Intent Classification + Slot Extraction)
→ State Update
→ Dialogue Manager
→ Database Query (Cambridge Restaurant Database)
→ Template Generator
→ System Response

The modular separation also ensures that each component has a well-defined task, as illustrated in Figure 4. The NLU module is responsible for extracting semantic information from free text, while the Dialogue Manager and State Update components are responsible for maintaining context information. The Database module is responsible for retrieving restaurant information based on slot values, while the Generator module is responsible for generating natural language responses based on predefined templates. This clear separation of responsibilities improves maintainability and allows individual modules to be modified or extended independently.

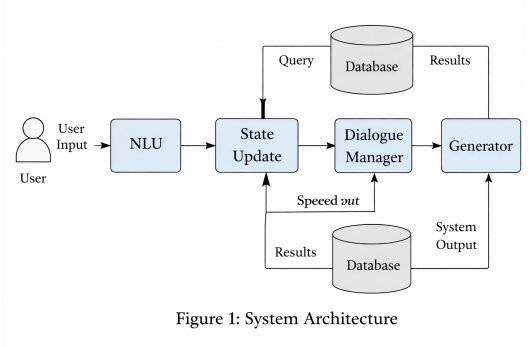


Figure 1: System Architecture

Figure 1: Modular architecture of the dialogue system showing NLU, Dialogue Manager, Database, and Generator components.

As illustrated in Figure 1, this separation of concerns improves maintainability and allows individual modules to be modified or extended independently.

### 3.2 Data Flow and Processing Steps

The processing of a user query proceeds through the following stages:

#### 3.2.1 Input Preprocessing

The raw user utterance is normalized through lowercasing and punctuation handling to ensure consistent pattern matching.

#### 3.2.2 Intent Classification

The NLU module evaluates predefined regular expression patterns sequentially to determine the user's intent. The first matching pattern defines the intent label.

#### 3.2.3 Entity Extraction (Slot Filling)

The system identifies relevant entities, including cuisine type, price range, and geographic area, using regular expression patterns and predefined vocabularies. Multiple entities may be extracted from a single query.

#### 3.2.4 State Update

Extracted slot values are passed to the dialogue state component, which maintains conversational context across turns.

#### 3.2.5 Database Query Formation

Based on the updated state, a structured query is generated to retrieve matching restaurants from the Cambridge restaurant database.

#### 3.2.6 Response Generation

Retrieved database results are inserted into predefined response templates, which are returned to the user. This pipeline enables the transformation of unstructured natural language input into structured database queries while preserving conversational continuity through dialogue state tracking.

## 4 Natural Language Understanding Module

The Natural Language Understanding (NLU) component is responsible for converting free-text input from the user into a structured representation consisting of an intent label and corresponding slot values. The implemented solution follows a deterministic rule-based approach based on regular expression pattern matching and predefined entity vocabularies. As illustrated in Figure 2, the NLU pipeline processes user input through sequential stages, including preprocessing, intent classification, and entity extraction, ultimately producing a structured output that can be consumed by the dialogue management component.

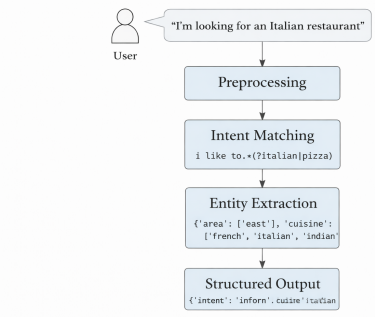


Figure 2: NLU Processing Flow

### 4.1 Preprocessing

Before the semantic analysis, the user input is normalized. This is done to ensure consistency in the

pattern matching process. The preprocessing stage involves the following:

Conversion of all characters to lowercase

Removal or handling of punctuation

Standardized formatting for matching operations

Normalization is particularly important in rule-based systems, where pattern matching is performed on the basis of exact lexical correspondence. Unlike statistical systems, stemming, lemmatization, or semantic embedding techniques are not applied. As illustrated in Figure 3, the preprocessing pipeline consists of lowercasing and punctuation removal to ensure consistent textual representation before intent matching. By transforming the user input into a normalized form, the system reduces variability caused by formatting differences and improves the reliability of deterministic regular expression matching.




Figure 3: Preprocessing Module.

## 4.2 Intent Classification

The system supports 13 predefined conversational intents, including greeting, information-seeking, alternative-seeking, and dialogue control actions. Intent identification is performed using regular expression patterns defined for each intent category. As illustrated in Figure 4, the classifier evaluates the preprocessed user input against these predefined patterns in a sequential manner. The first matching pattern determines the assigned intent label, following a deterministic first-match-wins strategy.

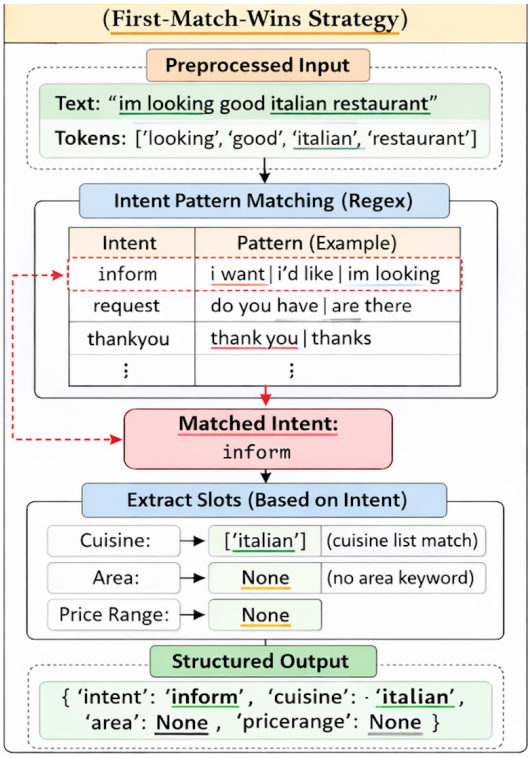



Figure 4: Intent Classification

The classification process adopts a sequential matching approach:

1. The normalized user input is compared against the pattern set of each intent.
2. Patterns are evaluated in predefined order.
3. The first matching pattern determines the intent label.
4. Once a match is found, the search terminates.

This approach can be summarized as:

Listing 1: Intent Classification Algorithm

```
for intent in intent_list:
    if regex_match(intent.pattern,
    user_input):
        return intent
```

The system is thus marked by the strategy of first match wins. Only one intention can be associated with a given user utterance. There are no confidence scores or ranking algorithms calculated.

### 4.2.1 Advantages

- Full transparency of decision rules
- Predictable system behavior

- No training data required
- Easy debugging and modification

#### 4.2.2 Limitations

- Sensitivity to linguistic variation
- Inability to handle multiple intents in a single utterance
- No probabilistic reasoning
- Dependence on manually defined patterns

Despite these limitations, the approach is well-suited for a domain-constrained restaurant recommendation task where user expressions follow relatively predictable patterns.

### 4.3 Entity Extraction and Slot Filling

In addition to intent detection, the NLU module performs entity extraction to identify relevant slot values required for database querying. The system extracts the following entities:

**Cuisine Type** (approximately 90 predefined categories)

**Price Range** (cheap, moderate, expensive)

**Geographic Area** (north, south, east, west, centre)

The detection of entities is done through regular expression matching and vocabulary lookup. Unlike the intent classifier, the entity extraction process supports multiple matches in a single utterance. This means that a user query can identify cuisine, price, and area at the same time.

For example: Input: "I am looking for a cheap Italian restaurant in the north." Output representation:

Intent: ask

Slots:

cuisine: italian

pricerange: cheap

area: north

The extracted slots are then fed to the dialogue state update component, which has the capability to store the conversation memory.

The entity recognition system has a closed vocabulary. Any word that is not defined in the pattern set is not recognized. There is no fuzzy matching, synonym expansion, or spelling correction done.

### 4.4 Design Rationale

The choice of implementing a rule-based approach was driven by the limited nature of the domain and the educational nature of the project. The restaurant recommendation task has a limited and pre-defined set of intents and entity values, which makes pattern-based matching possible and efficient.

In comparison to machine learning solutions, the developed system:

Does not require annotated training data

Provides complete transparency of decision rules

Allows straightforward debugging and modification

Ensures deterministic output for identical input

This design aligns with traditional task-oriented dialogue systems and demonstrates the effectiveness of rule-based NLU in structured domains.

## 5 Dialogue Management and Database Integration

Although the main achievement of this project is the implementation of the NLU module, the chatbot is based on a larger modular dialogue system. This section explains how the NLU module, the dialogue state update process, the restaurant database, and the response generation module interact with each other.

### 5.1 Dialogue State Management

The Dialogue State Management component is tasked with the responsibility of managing the conversation states. Once the NLU module has identified the intent and the corresponding slot values from the user's statement, these are passed to the State Update component.

The State Update mechanism:

Stores extracted slot values (e.g., cuisine, price range, area)

Updates previously stored values when new information is provided

Preserves contextual information across turns

Supports alternative requests and dialogue restart operations

Crucially, the NLU component itself does not have dialogue memory. Rather, it outputs a structured form of the current user input, which is then integrated into the dialogue state by another component. This division of labor follows standard

modular dialogue system design practice (Young et al., 2013).

For example, in a multi-turn interaction: User: "I am looking for an Italian restaurant." User: "In the north and cheap."

The NLU extracts relevant slot values from each turn independently, while the dialogue state component merges them into a unified structured query.

#### 5.1.1 Database Query Generation

The chatbot is connected to a structured database containing restaurant information for Cambridge. After the dialogue state has been updated with all necessary slot values, a database query is constructed dynamically.

The query is based on the extracted constraints:

- Cuisine type
- Price range
- Geographic area

If certain slots are unspecified, the query is generated using the available constraints. The database then returns matching restaurant entries.

This integration demonstrates how natural language input can be transformed into structured database queries through deterministic semantic parsing.

#### 5.1.2 Template-Based Response Generation

Once relevant restaurant records are retrieved from the database, the response generation component constructs a natural language reply using predefined templates.

The generator:

- Selects an appropriate response template based on intent
- Inserts retrieved database values into placeholder fields
- Produces the final textual response presented to the user

For example:

Template:

"There is a {pricerange} {cuisine} restaurant in the {area} called {name}."

Filled Response:

"There is a cheap Italian restaurant in the north called Roma Kitchen."

This template-based generation approach ensures grammatical consistency and avoids unpredictable output behavior. Since the system operates within a constrained domain, predefined templates are sufficient to produce informative and coherent responses.

The modular interaction between NLU, state management, database querying, and response generation enables the system to support structured information retrieval through natural language dialogue.

## 6 Functional Testing and Validation

The developed system was tested using systematic functional testing. The NLU module is deterministic and rule-based. Therefore, the testing of the system was done by ensuring that it functions as expected in various conversation scenarios.

### 6.1 Testing Methodology

The validation process involved manually designed test cases covering:

- All 13 defined intents
- Single-slot and multi-slot queries
- Alternative requests
- Dialogue restart scenarios
- Information requests (phone, address, postcode, details)
- Greeting and closing interactions

Each test case consisted of a user utterance and the expected structured output, including detected intent and extracted slot values. The system's output was compared against the expected result to verify correctness.

Because the classification mechanism relies on exact regular expression matching, the evaluation emphasized coverage of pattern definitions and verification of preprocessing behavior (e.g., lowercasing and punctuation handling).

### 6.2 Representative Test Scenarios

Several representative scenarios are described below.

#### 6.2.1 Scenario 1: Greeting Detection

Input:

"Hello"

Expected Output:

Intent: greet

The system correctly identifies greeting expressions through predefined regular expression patterns.

#### 6.2.2 Scenario 2: Multi-Slot Query

Input:

"I am looking for a cheap Chinese restaurant in the south."

Expected Output:

Intent: ask

Slots:

- pricerange: cheap
- cuisine: chinese
- area: south

The system successfully extracts multiple entities within a single utterance and forwards them for database querying.

#### 6.2.3 Scenario 3: Alternative Request

Input:

"Do you have another option?"

Expected Output:

Intent: request_alternative

This verifies the detection of dialogue control intents.

#### 6.2.4 Scenario 4: Information Request

Input:

"What is the phone number?"

Expected Output:

Intent: inform.phone

This confirms correct classification of detail-specific requests.

#### 6.2.5 Scenario 5: Restart Operation

Input:

"Start over."

Expected Output:

Intent: restart

This ensures that conversation control commands are properly identified.

### 6.3 Discussion of Evaluation Approach

However, no quantitative measures of accuracy, precision, or F1 score were calculated, as the system does not employ probabilistic classification. Rather, correctness is determined by exact rule matching.

The deterministic approach of the implementation ensures that the same input will always generate the same output. Hence, the evaluation mainly focuses on rule coverage and robustness in the specified conversational domain.

The functional testing showed that the NLU module correctly identifies the intent and extracts the slot values for queries that match the specified pattern set.

## 7 Challenges and Limitations

Although the implemented NLU module performs reliably within its defined scope, several challenges and limitations arise from the deterministic rule-based design.

### 7.1 Linguistic Variability and Synonym Handling

The approach is based on pre-defined regular expression patterns and vocabularies. Therefore, the approach is only able to identify words and phrases that have been explicitly defined in the patterns. Synonyms or paraphrases that have not been explicitly defined are not identified.

For instance, although the word "cheap" may be identified as a price range, other phrases such as "low-cost" or "budget-friendly" are not identified unless they have been explicitly defined in the pattern set (Jurafsky and Martin, 2023).

### 7.2 Sensitivity to Input Variations

Because the classification process relies on exact pattern matching, the system is intolerant of spelling mistakes and unexpected word forms. Spelling mistakes, unexpected vocabulary, or unconventional wording can impede the detection of correct intent or entities.

Unlike machine learning algorithms, the system does not generalize beyond its predefined rules. Although preprocessing is done to normalize casing and punctuation, stemming, lemmatization, or fuzzy matching is not employed.

### 7.3 First-Match Intent Strategy

The intent classifier follows a sequential first-match-wins approach. As a result:

- Only one intent can be assigned per utterance.
- If multiple patterns could potentially match, only the first detected intent is returned.
- No ranking or confidence estimation is performed.

This design simplifies implementation but may lead to unintended behavior in ambiguous cases.

### 7.4 Single-Intent Constraint

The system does not support multi-intent utterances. Queries that contain multiple functional goals in a single sentence cannot be fully represented. For example:

"Find a cheap Italian restaurant and tell me its address."

The above input contains a search request and an information request. However, the current design can only process one intent at a time.

### 7.5 Scalability

The rule-based system grows linearly with the number of patterns and entities. With the growth of the domain, the number of regular expressions rises, making it harder to maintain and possibly leading to conflicts between patterns.

On the other hand, data-driven systems are able to generalize over larger linguistic differences once they have been trained on enough data (Young et al., 2013).

### 7.6 Domain Dependency

The implemented NLU module is tightly coupled to the restaurant recommendation domain. The predefined entity sets (e.g., cuisine types, price ranges, geographic areas) are specific to this application. Adapting the system to a different domain would require manual redefinition of patterns and slot vocabularies. Despite these limitations, the rule-based approach provides transparency, deterministic behavior, and ease of debugging, making it appropriate for constrained task-oriented dialogue systems in controlled environments.

## 8 Conclusion and Future Work

This project involved the design and development of a rule-based Natural Language Understanding (NLU) component for a domain-specific restaurant recommendation chatbot. The system converts free-text user input into formal semantic representations in the form of an intent label and corresponding slot values. The design is based on a modular pipeline architecture, which segregates NLU processing, dialogue state management, database querying, and template-based response generation.

The NLU component uses deterministic regular expression pattern matching for intent identification and entity extraction. The system is capable of handling 13 predefined conversational intents and about 90 cuisine types, with price range and geographic region restrictions. Multiple entities can be extracted from a single user input, allowing for structured database querying based on user-specified parameters. Functional validation has shown the system to work correctly in greeting, information retrieval, alternative request, and dialogue control tasks in the restricted domain.

The deterministic approach of the implementation ensures transparency and predictability, making it suitable for controlled task-oriented dialogue settings. Nevertheless, some weaknesses of the system were also discovered, including sensitivity to linguistic variation, the absence of synonym generalization, the lack of confidence scoring, and the limitation to single-intent statements.

There are several ways that the system could be further developed in the future. Firstly, the integration of statistical or neural approaches to intent classification could make the system more robust to paraphrasing and novel expressions. Secondly, the integration of fuzzy matching or synonym expansion could make the system more flexible in entity recognition. Thirdly, the extension of the system to multi-intent detection and advanced dialogue state reasoning could make it more suitable for complex user dialogues. Finally, the evaluation of the system could be further expanded to include quantitative performance measures.

Overall, this project demonstrates that rule-based NLU remains a viable and effective approach for domain-constrained conversational systems, particularly in scenarios where transparency, simplicity, and controlled behavior are prioritized.

## Acknowledgements

The author would like to thank Dr. Stefan Ultes for his supervision, guidance, and valuable feedback throughout the development of this project. His expertise in dialogue systems and natural language processing greatly contributed to the successful completion of this work

## Declaration of AI Tool Use

Tools used: ChatGPT (OpenAI).

Purposes: Support in structuring the report, refinement of academic language, drafting section formulations, assistance with figure captions and LaTeX formatting.

## References


Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186.

Daniel Jurafsky and James H. Martin. 2023. *Speech and Language Processing*, 3rd edition. Draft version available online.

Joseph Weizenbaum. 1966. Eliza—a computer program for the study of natural language communication between man and machine. *Communications of the ACM*, 9(1):36–45.

Steve Young, Milica Gasic, Blaise Thomson, and Jason D. Williams. 2013. Pomdp-based statistical spoken dialogue systems: A review. *Proceedings of the IEEE*, 101(5):1160–1179.